SAMURAI ROBOT

Система автономного управления

Полный технический отчёт

Архитектура: от постановки задачи до движения моторов

Платформа:	Гусеничный робот (Raspberry Pi 4B)
ОС Pi:	Debian 13 Trixie (pure Python)
ОС лэптоп:	Ubuntu + ROS2 Humble (Docker)
Связь:	MQTT (Pi) ↔ ROS2 (лэптоп)
Задача:	Автономная охота за мячом
Детектор:	YOLOv11n + HSV-классификация
Навигация:	Log-odds SLAM + A* + Bug0

Уровень	Ключевые компоненты
1. Постановка задачи	FSM (10 состояний), голос, жесты, миссии
2. Навигация	Nav2/ROS2, SLAM, A*, Bug0
3. Оценка состояния	IMU EKF, Velocity EKF, Accel-fusion
4. Локальное упр-е	PI-контроллер курса + путевой регулятор
5. Безопасность	Collision Guard, fallback_nav, ZUPT
6. Исполнение	MotorNode → PCA9685 → PWM

Версия 1.0 | Апрель 2026 | Самурай Проект

Содержание

1 Обзор системной архитектуры 2
- 1.1 Концепция трёхуровневого разделения . . . 2
- 1.2 Поток данных (сигнальный граф) . . . 2
- 1.3 Топология MQTT-тем . . . 2

2 Блок 1: Постановка задачи 3
- 2.1 Конечный автомат (FSM) . . . 3
  - 2.1.1 Граф состояний . . . 3
  - 2.1.2 Алгоритм захвата мяча (последовательность GRABBING) . . . 3
  - 2.1.3 Логика наведения на мяч . . . 3
- 2.2 Голосовое управление (русский язык) . . . 4
- 2.3 Жестовое управление . . . 4
- 2.4 Миссионное планирование (mission_node) . . . 4

3 Блок 2: Навигация и планирование 5
- 3.1 Запись и воспроизведение пути (path_recorder_node) . . . 5
  - 3.1.1 Критерий записи точки . . . 5
  - 3.1.2 Воспроизведение (Pure Pursuit) . . . 5
- 3.2 Nav2 / ROS2 Humble . . . 5
- 3.3 A* в симуляторе . . . 5
  - 3.3.1 Эвристика (Octile distance) . . . 6
  - 3.3.2 Стоимость перехода . . . 6
  - 3.3.3 Расширение препятствий (Configuration Space) . . . 6
- 3.4 Bug0 (аварийный планировщик) . . . 6

4 Блок 3: Оценивание состояния 7
- 4.1 IMU-EKF (ekf_imu.py) . . . 7
  - 4.1.1 Вектор состояния . . . 7
  - 4.1.2 Модель движения (Prediction step) . . . 7
  - 4.1.3 ZUPT — Zero-velocity Update (коррекция нуля) . . . 7
  - 4.1.4 Крен и тангаж из акселерометра . . . 8
- 4.2 Velocity EKF (velocity_ekf.py) . . . 8
  - 4.2.1 Два параллельных 2D-фильтра . . . 8
  - 4.2.2 Модель двигателя (Prediction step) . . . 8
  - 4.2.3 Обновление по акселерометру . . . 9
  - 4.2.4 ZUPT для скорости . . . 9
  - 4.2.5 Неголономная связь . . . 9
  - 4.2.6 Пересентрирование смещения (Anti-drift) . . . 9
- 4.3 Оценка положения из акселерометра (accel_position.py) . . . 9
  - 4.3.1 Каскад фильтрации . . . 9
  - 4.3.2 Удаление гравитации (Body → World) . . . 9
  - 4.3.3 2-й порядок IIR (каскадный LPF) . . . 9
  - 4.3.4 Адаптивный масштаб колёс . . . 10
- 4.4 Полная одометрия (motor_node.py) . . . 10
  - 4.4.1 Интегрирование положения . . . 10

4.4.2 Слияние скоростей . . . 10
4.4.3 Публикуемое состояние @ 20 Гц . . . 10

5 Блок 4: Локальное управление 11
5.1 Precision Drive (precision_drive_node.py) . . . 11
5.1.1 Профиль скорости . . . 11
5.1.2 PI-регулятор курса . . . 11
5.1.3 Боковой корректор (Lateral correction) . . . 11
5.1.4 Алгоритм точного поворота . . . 11
5.1.5 Сценарии испытаний . . . 12
5.1.6 Обнаружение и обработка помех . . . 12
5.2 Дифференциальный привод (motor_driver.py) . . . 12

6 Блок 5: Безопасность 13
6.1 Collision Guard (motor_node.py) . . . 13
6.1.1 Зоны защиты . . . 13
6.1.2 Приоритет команд . . . 13
6.2 Fallback Navigation (fallback_nav_node) . . . 13
6.3 Watchdog Node (watchdog_node.py) . . . 13
6.4 ZUPT как функция безопасности . . . 14
6.5 Обнаружение препятствий и обновление карты . . . 14

7 Блок 6: Исполнительный уровень 15
7.1 Архитектура исполнительного уровня . . . 15
7.2 PCA9685 — ШИМ-контроллер (pca9685_driver.py) . . . 15
7.2.1 Параметры ШИМ . . . 15
7.2.2 Преобразование дроссель → ШИМ . . . 15
7.2.3 Назначение каналов . . . 15
7.3 Servo Driver (servo_driver.py) . . . 16
7.3.1 Преобразование угол → длительность импульса . . . 16
7.3.2 Авто-отключение (Anti-buzz) . . . 16
7.4 LED Driver (led_driver.py) . . . 16

8 Датчики 17
8.1 IMU — MPU6050 (imu_node.py) . . . 17
8.1.1 Калибровка смещений . . . 17
8.1.2 EMA фильтрация . . . 17
8.2 Ультразвуковой дальномер HC-SR04 (ultrasonic_node) . . . 17
8.2.1 Принцип измерения . . . 17
8.3 Камера CSI (camera_node) . . . 17
8.3.1 Моно-оценка дальности до объекта . . . 17

9 SLAM и картография (slam_map_node.py) 19
9.1 Log-odds occupancy grid . . . 19
9.1.1 Обновление ячеек . . . 19
9.1.2 Порог занятости . . . 19
9.2 Трассировка лучей (Ray casting, HC-SR04) . . . 19
9.3 Реестр объектов . . . 19

10 Система восприятия (yolo_detector_node.py) 20
10.1 YOLOv11n — архитектура и параметры 20
10.2 Конвейер инференса 20
10.3 HSV-классификация цвета мяча 20

11 Сводная таблица параметров системы 21

12 Топология MQTT-топиков 23

13 Взаимодействие уровней: полный цикл управления 24

14 Заключение 25

1. Обзор системной архитектуры

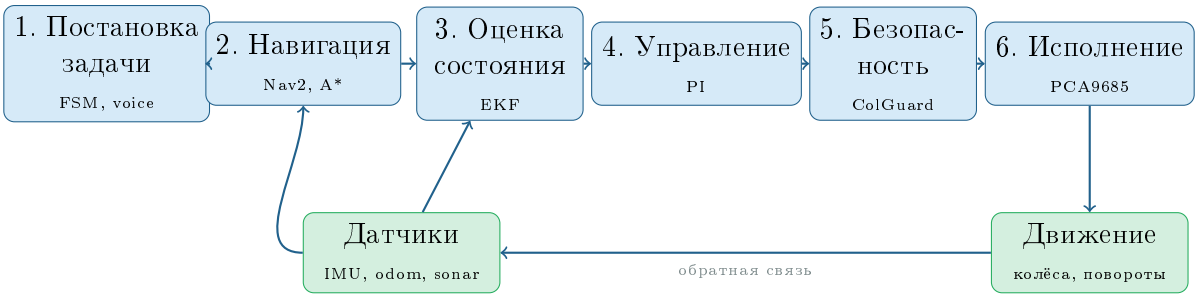
1.1. Концепция трёхуровневого разделения

Система Samurai реализует трёхуровневую гетерогенную архитектуру, разделённую по вычислительным возможностям узлов:

Три уровня системы

- **Pi (робот)** Pure Python 3 + paho-mqtt. Нет ROS2, нет Docker. Все задачи реального времени: моторы, датчики, FSM, SLAM, фильтрация.
- **Лэптоп** ROS2 Humble в Docker. YOLO-детекция, Nav2, FastAPI-дашборд. Подключается по Wi-Fi к брокеру Mosquitto на Pi.
- **Android** Kotlin+Compose. REST + SocketIO + MQTT. Offline-распознавание речи (Vosk).

1.2. Поток данных (сигнальный граф)



1.3. Топология MQTT-тем

Все узлы обмениваются через брокер Mosquitto под единым пространством имён:

samurai/{robot_id}/{subsystem}/{topic}

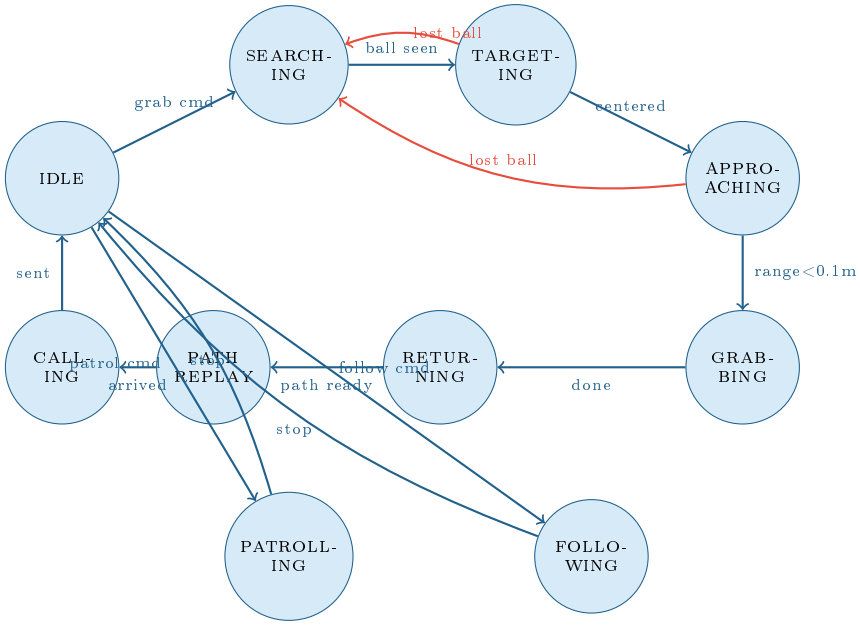
Базовый класс MqttNode инкапсулирует подключение, автоподписку, QoS, retain-флаги и мониторинг производительности (warn > 20 мс на callback, error > 200 мс на таймер).

2. Блок 1: Постановка задачи

2.1. Конечный автомат (FSM)

Центральный управляющий элемент — fsm_node.py — реализует детерминированный конечный автомат (Mealy-машина) с 10 состояниями.

2.1.1. Граф состояний



2.1.2. Алгоритм захвата мяча (последовательность GRABBING)

Временная диаграмма GRABBING

$$t \in [0,\ 1\ \text{с}]: \quad v_{lin} = 0.05\ \text{м/с}, \quad \theta_{claw} = 0^\circ\ (\text{открыть})$$

$$t \in [1,\ 2\ \text{с}]: \quad v_{lin} = 0, \quad \theta_{claw} = 0^\circ$$

$$t \in [2,\ 3\ \text{с}]: \quad v_{lin} = 0, \quad \theta_{claw} = 90^\circ\ (\text{закрыть})$$

$$t > 3\ \text{с}: \quad \text{переход} \rightarrow \text{RETURNING}$$

2.1.3. Логика наведения на мяч

Фаза TARGETING (угловое выравнивание). Ошибка по горизонтали нормируется относительно ширины кадра (640 пикс.):

$$e_x = \frac{x_{ball} + w_{ball}/2 - x_{img,center}}{x_{img,center}}, \quad x_{img,center} = 320\ \text{пикс.} \tag{1}$$

Угловая скорость с первичным LP-фильтром:

$$\omega_{target} = -e_x \cdot K_{steer}, \quad K_{steer} = 0.8\ \text{рад/с} \tag{2}$$

$$\omega_{cmd}[k] = 0.4\,\omega_{cmd}[k-1] + 0.6\,\omega_{target}[k] \tag{3}$$

Переход в APPROACHING: $|e_x| < 0.15$.

Фаза APPROACHING (линейное сближение).

$$v_{lin} = \begin{cases} 0.12\ \text{м/с} & |e_x| < 0.15 \\ 0.06\ \text{м/с} & \text{иначе} \end{cases} \tag{4}$$

Переход в GRABBING: $r_{sonar} < 0.10\,\text{м}$.

2.2. Голосовое управление (русский язык)

Команды распознаются regex-паттернами по топику samurai/{id}/voice_command:

Паттерн (regex)	Действие	Переход FSM
получивозьми|найди|	Захват мяча	IDLE → SEARCHING
стопостановись|стой|	Стоп	$*$ → IDLE
домойвернись|	Домой	$*$ → RETURNING
патрулобход|	Патрулирование	IDLE → PATROLLING
следуйза мной|	Слежение	IDLE → FOLLOWING
запиши путь	Запись пути	—
воспроизведи путь	Воспроизведение	—
вперёдпрямо|	Движение вперёд	—
назад	Движение назад	—
налевовлево|	Поворот влево	—
направовправо|	Поворот вправо	—

2.3. Жестовое управление

Узел YOLO-детектора публикует результаты распознавания жестов в топик samurai/{id}/gesture/c
FSM подписывается и обрабатывает события аналогично голосовым командам.

2.4. Миссионное планирование (mission_node)

Опциональный узел mission_node реализует последовательное выполнение задач из YAML-конфига:

1. goto — перемещение в точку (через goal_pose)
2. grab — запуск FSM-захвата
3. patrol — патрулирование маршрута
4. wait — ожидание события

3. Блок 2: Навигация и планирование

3.1. Запись и воспроизведение пути (path_recorder_node)

Простейший метод навигации — запись одометрических поз при движении и воспроизведение их в обратном порядке (возврат домой).

3.1.1. Критерий записи точки

Новая поза записывается при выполнении хотя бы одного из условий:

$$\Delta s = \sqrt{(x - x_p)^2 + (y - y_p)^2} > d_{rec} = 0.025 \text{ м} \tag{5}$$

$$\Delta\theta = |\theta - \theta_p| > \alpha_{rec} = 0.06 \text{ рад} \approx 3.4^\circ \tag{6}$$

3.1.2. Воспроизведение (Pure Pursuit)

Путь воспроизводится с методом Pure Pursuit с переменной точкой упреждения:

$$L_d = 0.08 \text{ м} \quad \text{(lookahead distance)} \tag{7}$$

Алгоритм на каждом шаге:

1. Найти ближайшую точку пути $\mathbf{w}_i$.
2. Найти точку упреждения $\mathbf{w}_{ld}$ на расстоянии $\geq L_d$ от робота.
3. Вычислить угол до $\mathbf{w}_{ld}$ в системе робота:

$$\alpha = \text{atan2}(w_{ld,y} - y,\ w_{ld,x} - x) - \theta$$

4. Команда:

$$v_{cmd} = v_{replay} = 0.18 \text{ м/с}, \quad \omega_{cmd} = \frac{2v_{cmd}\sin\alpha}{L_d}$$

3.2. Nav2 / ROS2 Humble

На стороне лэптопа поднимается полный стек навигации ROS2:

- slam_toolbox — онлайн SLAM с лазером (или симулированными scans)
- nav2_bt_navigator — Behaviour Tree навигатор
- nav2_dwb_controller — DWB локальный планировщик
- nav2_navfn_planner — Navfn глобальный планировщик (Dijkstra/A*)

Команда цели публикуется из FSM: samurai/{id}/goal_pose → /goal_pose (PoseStamped) через MQTT-мост.

3.3. A* в симуляторе

Симулятор (simulator.py) использует самостоятельную реализацию A* на сетке 200×200 (разрешение 0.05 м):

3.3.1. Эвристика (Octile distance)

$$h(n) = \max(\Delta r, \Delta c) + (\sqrt{2} - 1) \cdot \min(\Delta r, \Delta c) \approx \max(\Delta r, \Delta c) + 0.414 \cdot \min(\Delta r, \Delta c) \quad (8)$$

Это точная нижняя оценка для 8-связной сетки с диагональным шагом.

3.3.2. Стоимость перехода

$$g(n, n') = \begin{cases} 1.0 & \text{ортогональный шаг} \\ \sqrt{2} \approx 1.414 & \text{диагональный шаг} \end{cases} \quad (9)$$

3.3.3. Расширение препятствий (Configuration Space)

Перед поиском препятствия расширяются на марж m:

$$m = r_{robot} + r_{safety} = 0.12 + 0.10 = 0.22 \text{ м} \quad \Rightarrow \quad 5 \text{ ячеек} \quad (10)$$

Операция: binary_dilation(obstacle_mask, iterations=5).

3.4. Bug0 (аварийный планировщик)

Алгоритм Bug0 используется как fallback_nav_node при недоступности Nav2:

1. Двигаться к цели по прямой.
2. При обнаружении препятствия ($r_{sonar} < r_{thresh}$): следовать вдоль его границы (поворот на 90°, движение вдоль стены).
3. Когда направление на цель совпадает с текущим курсом и путь свободен: перейти в шаг 1.

4. Блок 3: Оценивание состояния

Состояние робота полностью описывается вектором:

$$\mathbf{s} = \begin{bmatrix} x, & y, & \theta, & v, & \omega \end{bmatrix}^{\mathsf{T}} \tag{11}$$

где (x, y) — положение (м), θ — курс (рад), v — линейная скорость (м/с), ω — угловая скорость (рад/с).

4.1. IMU-EKF (ekf_imu.py)

4.1.1. Вектор состояния

Фильтр работает в плоскости (ground robot), оценивая только курс и смещение гироскопа:

$$\mathbf{x}_{imu} = \begin{bmatrix} \psi \\ b_z \end{bmatrix} \tag{12}$$

где ψ — рыскание (yaw, рад), b_z — смещение гироскопа по оси Z (рад/с).

4.1.2. Модель движения (Prediction step)

$$\hat{\psi}_{k|k-1} = \psi_{k-1} + (\omega_z^{meas} - b_z) \cdot \Delta t \tag{13}$$

$$\hat{b}_{z,k|k-1} = b_z \quad \text{(случайное блуждание)} \tag{14}$$

Матрица перехода и ковариации процесса:

$$F = \begin{bmatrix} 1 & -\Delta t \\ 0 & 1 \end{bmatrix}, \quad Q = \begin{bmatrix} q_\psi \Delta t & 0 \\ 0 & q_b \Delta t \end{bmatrix} \tag{15}$$

$$q_\psi = 0.001 \text{ рад}^2, \quad q_b = 0.0001 \text{ (рад/с)}^2 \tag{16}$$

Предсказание ковариации:

$$P_{k|k-1} = F\, P_{k-1}\, F^{\mathsf{T}} + Q \tag{17}$$

4.1.3. ZUPT — Zero-velocity Update (коррекция нуля)

При стационарном состоянии ($|\omega_z^{corrected}| < \varepsilon_{enter}$) применяется Kalman-коррекция, используя условие $\omega_z = b_z$ как измерение:

ZUPT: условие и коррекция

Гистерезис:

$$\text{stationary} = \begin{cases} \text{True} & |\omega_z - b_z| < \varepsilon_{enter} = 0.015 \text{ рад/с}, \quad N \geq 5 \\ \text{False} & |\omega_z - b_z| > \varepsilon_{exit} = 0.04 \text{ рад/с} \end{cases}$$

Измерение: $z = \omega_z^{meas}$, $H = [0,\ 1]$ (измеряем смещение)

$$S = P_{bb} + r_{zupt}, \quad r_{zupt} = 10^{-4}$$
$$K_\psi = P_{\psi b}/S, \quad K_b = P_{bb}/S$$
$$\psi \mathrel{+}= K_\psi \cdot (\omega_z - b_z), \quad b_z \mathrel{+}= K_b \cdot (\omega_z - b_z)$$

4.1.4. Крен и тангаж из акселерометра

$$\phi = \arctan\left(\frac{a_y}{a_z}\right) - \phi_0, \quad \vartheta = \arctan\left(\frac{-a_x}{\sqrt{a_y^2 + a_z^2}}\right) - \vartheta_0 \tag{18}$$

Ворота акселерометра (gate): обновление пропускается, если:

$$\Big| \|\mathbf{a}\| - g \Big| > \gamma \cdot g, \quad \gamma = 0.30 \tag{19}$$

4.2. Velocity EKF (velocity_ekf.py)

4.2.1. Два параллельных 2D-фильтра

Для осей X и Y работают независимые фильтры с одинаковой структурой:

$$\mathbf{x}_v = \begin{bmatrix} v_x \\ b_{ax} \end{bmatrix}, \quad \mathbf{x}_{vy} = \begin{bmatrix} v_y \\ b_{ay} \end{bmatrix} \tag{20}$$

где b_{ax}, b_{ay} — смещения акселерометра.

4.2.2. Модель двигателя (Prediction step)

Динамика первого порядка (lag-модель):

$$\tau = \begin{cases} \tau_{brake} = 0.08 \text{ с} & v_{cmd} \cdot v < 0 \ \text{(торможение)} \\ \tau_{motor} = 0.25 \text{ с} & \text{иначе} \end{cases} \tag{21}$$

$$\alpha = \min\left(\frac{\Delta t}{\tau},\ 1.0\right) \tag{22}$$

$$\hat{v}_{k|k-1} = v_k + (v_{cmd} - v_k) \cdot \alpha \tag{23}$$

Якобиан предсказания (линеаризация):

$$f_{vv} = 1 - \alpha, \quad f_{vb} = 0, \quad f_{bv} = 0, \quad f_{bb} = 1 \tag{24}$$

$$P_{k|k-1} = \begin{bmatrix} f_{vv}^2 P_{vv} + q_v \Delta t & f_{vv} P_{vb} \\ f_{vv} P_{bv} & P_{bb} + q_b \Delta t \end{bmatrix} \tag{25}$$

$$q_v = 0.20, \quad q_b = 0.003 \tag{26}$$

4.2.3. Обновление по акселерометру

Модель измерения: акселерометр интегрирует скорость + смещение:

$$z_{acc} = v + b + \eta_{acc}, \quad H = [1,\ 1] \tag{27}$$

$$\nu = z_{acc} - \hat{v} - \hat{b}, \quad S = P_{vv} + P_{vb} + P_{bv} + P_{bb} + r_{acc} \tag{28}$$

$$K_v = \frac{P_{vv} + P_{vb}}{S}, \quad K_b = \frac{P_{bv} + P_{bb}}{S} \tag{29}$$

$$v \mathrel{+}= K_v \nu, \quad b \mathrel{+}= K_b \nu, \quad r_{acc} = 0.06 \tag{30}$$

4.2.4. ZUPT для скорости

$$H_{zupt} = [1,\ 0], \quad z_{zupt} = 0 \tag{31}$$

$$S_{zupt} = P_{vv} + r_{zupt}, \quad r_{zupt} = 10^{-4} \tag{32}$$

$$K_v = P_{vv}/S_{zupt}, \quad K_b = P_{bv}/S_{zupt} \tag{33}$$

$$v \mathrel{+}= K_v \cdot (-v), \quad b \mathrel{+}= K_b \cdot (-v) \tag{34}$$

4.2.5. Неголономная связь

Гусеничный робот не может двигаться в боковом направлении:

$$v_y[k] \leftarrow v_y[k] \cdot (1 - \lambda_{lat}), \quad \lambda_{lat} = 0.15 \tag{35}$$

4.2.6. Пересentрирование смещения (Anti-drift)

Каждые 500 обновлений ($\approx 10\,$с при $50\,$Гц):

$$v_{acc,x} \mathrel{-}= b_x, \quad v_{acc,y} \mathrel{-}= b_y, \quad b_x = b_y = 0 \tag{36}$$

4.3. Оценка положения из акселерометра (accel_position.py)

4.3.1. Каскад фильтрации

$$\mathbf{a}_{raw} \xrightarrow{\text{Median spike reject}} \mathbf{a}_{med} \xrightarrow{\text{2nd-order IIR LPF}} \mathbf{a}_{filt} \xrightarrow{\text{Noise gate}} \mathbf{a}_{clean} \xrightarrow{\text{VelEKF}} \hat{v} \xrightarrow{\int} \hat{x} \tag{37}$$

4.3.2. Удаление гравитации (Body → World)

Матрица поворота через углы Эйлера (ϕ, ϑ, ψ):

$$R = R_z(\psi)\, R_y(\vartheta)\, R_x(\phi) \tag{38}$$

$$\mathbf{a}_{world} = R\,\mathbf{a}_{body} - \mathbf{g}_0, \quad \mathbf{g}_0 = R\,\mathbf{g}_{calibrated} \tag{39}$$

4.3.3. 2-й порядок IIR (каскадный LPF)

Два последовательных звена первого порядка:

$$s_1[k] = s_1[k-1] + \alpha\Big(x[k] - s_1[k-1]\Big) \tag{40}$$

$$s_2[k] = s_2[k-1] + \alpha\Big(s_1[k] - s_2[k-1]\Big) \tag{41}$$

Параметр $\alpha = 0.4$ соответствует частоте среза $\approx 2.2\,$Гц при $50\,$Гц дискретизации.

4.3.4. Адаптивный масштаб колёс

Онлайн-калибровка коэффициента колёс через сравнение с акселерометром:

$$s[k] = s[k-1] + \mu \left(\frac{\Delta x_{acc}}{\Delta x_{wheel}} - s[k-1] \right), \quad s \in [0.5,\ 2.0] \tag{42}$$

4.4. Полная одометрия (motor_node.py)

4.4.1. Интегрирование положения

$$x[k] = x[k-1] + v_x \cdot \Delta t \tag{43}$$

$$y[k] = y[k-1] + v_y \cdot \Delta t \tag{44}$$

$$\theta[k] = \psi_{imu} \quad \text{(из IMU-EKF, приоритет)} \tag{45}$$

4.4.2. Слияние скоростей

$$v_{x,fused} = \hat{v}_{accel} \cos\theta + \hat{v}_{y,accel} \sin\theta \tag{46}$$

Если акселерометрный оценщик недоступен — используются команды cmd_vel.

4.4.3. Публикуемое состояние @ 20 Гц

$$\mathbf{s}_{odom} = \Big[x,\ y,\ \theta,\ v_x,\ \omega_z,\ a_x,\ a_y,\ \text{stationary},\ \text{wheel_scale}\Big]^{\mathsf{T}} \tag{47}$$

5. Блок 4: Локальное управление

5.1. Precision Drive (precision_drive_node.py)

Узел реализует высокоточное прямолинейное движение и развороты с комбинацией курсового PI-регулятора и бокового корректора.

5.1.1. Профиль скорости

Трапецеидальный профиль скорости в зависимости от остатка дистанции d_{rem}:

$$v_{cmd} = \begin{cases} v_{drive} = 0.12 \text{ м/с} & d_{rem} > d_{slow} = 0.08 \text{ м} \\ v_{slow} = 0.06 \text{ м/с} & 0.02 < d_{rem} \leq 0.08 \text{ м} \\ v_{crawl} = 0.04 \text{ м/с} & d_{rem} \leq 0.02 \text{ м} \end{cases} \tag{48}$$

Критерий прибытия: $d_{rem} < 5\,\text{мм}$ в течение 3 последовательных тиков.

5.1.2. PI-регулятор курса

$$e_\theta[k] = \theta_{target} - \theta[k] \tag{49}$$

$$I_\theta[k] = \text{clamp}\Big(I_\theta[k-1] + e_\theta[k] \cdot \Delta t,\ -0.3,\ +0.3\Big) \tag{50}$$

$$u_\theta[k] = \text{clamp}\Big(K_P\, e_\theta + K_I\, I_\theta,\ -0.4,\ +0.4\Big) \text{ рад/с} \tag{51}$$

$$K_P = 2.5, \quad K_I = 0.3 \tag{52}$$

5.1.3. Боковой корректор (Lateral correction)

Боковое отклонение от линии движения:

$$e_{lat} = -(x - x_0)\sin\theta_0 + (y - y_0)\cos\theta_0 \tag{53}$$

где (x_0, y_0, θ_0) — поза начала манёвра.

$$u_{lat} = \text{clamp}\Big(-K_{lat}\, e_{lat},\ -0.3,\ +0.3\Big) \tag{54}$$

$$\omega_{cmd} = u_\theta + u_{lat} \tag{55}$$

5.1.4. Алгоритм точного поворота

$$e_\psi[k] = \psi_{target} - \psi[k] \tag{56}$$

$$\omega_{cmd} = \begin{cases} \text{sgn}(e_\psi) \cdot \omega_{slow} & |e_\psi| > \varepsilon_{turn,fast} \\ \text{sgn}(e_\psi) \cdot \omega_{crawl} & |e_\psi| \leq \varepsilon_{turn,fast} \end{cases} \tag{57}$$

Критерий завершения: $|e_\psi| < 0.02\,\text{рад}$ ($\approx 1.1°$).

5.1.5. Сценарии испытаний

Сценарий	Последовательность
cross	вперёд d, $+90°$, вперёд d, -90ř, вперёд d
square	[вперёд d, $+90°$] $\times$ 4
line	вперёд d с промежуточными коррекциями курса
zigzag	[вперёд 45°, поворот, назад 45°, поворот] $\times N$

5.1.6. Обнаружение и обработка помех

Удар/толчок (Push detection):

$$|a_y| > 3.0 \text{ м/с}^2 \;\Rightarrow\; \text{пауза, перестройка курса, продолжение} \tag{58}$$

Подъём (Lift detection):

$$\|\mathbf{g}\|_{body} < 6.0 \text{ м/с}^2 \text{ за 5 отсчётов} \;\Rightarrow\; \text{STOP (робот в воздухе)} \tag{59}$$

$$\text{Возобновление:} \quad \|\mathbf{g}\|_{body} > 8.5 \text{ м/с}^2 \tag{60}$$

5.2. Дифференциальный привод (motor_driver.py)

Базовый микшер для гусеничного шасси:

$$\ell = v_{lin} + \omega_{ang}, \quad r = v_{lin} - \omega_{ang} \tag{61}$$

$$\text{throttle}_{L,R} = \text{clamp}\left(\frac{\ell, r}{\max(|\ell|, |r|, 1)},\ -1.0,\ +1.0\right) \tag{62}$$

Мёртвые зоны: $|v_{lin}| < 0.01$ м/с и $|\omega| < 0.05$ рад/с обнуляются.

6. Блок 5: Безопасность

6.1. Collision Guard (motor_node.py)

Collision Guard — встроенный в motor_node аварийный ограничитель скорости на основе данных УЗ-дальномера.

6.1.1. Зоны защиты

Алгоритм Collision Guard

Зона полной остановки ($r < r_{stop} = 0.20$ м):

$$v_{lin} = 0, \quad \omega_{cmd} = \mathrm{sgn}(\omega_{steer}) \cdot 0.6 \text{ рад/с}$$

Робот поворачивается, объезжая препятствие.
Зона замедления ($r_{stop} \leq r < r_{warn} = 0.40$ м):

$$\lambda(r) = \frac{r - r_{stop}}{r_{warn} - r_{stop}} \in [0,\ 1]$$

$$v_{lin,limited} = v_{lin,cmd} \cdot \lambda(r)$$

Линейное замедление пропорционально близости препятствия.
Зона свободного движения ($r \geq r_{warn}$):

$$v_{lin,limited} = v_{lin,cmd}$$

6.1.2. Приоритет команд

Приоритет	Источник	Топик	Таймаут
1 (высший)	Ручной (джойстик/голос)	cmd_vel/manual	0.5 с
2	Collision override	внутренний	—
3 (низший)	Автономный (FSM/Nav)	cmd_vel	0.5 с

6.2. Fallback Navigation (fallback_nav_node)

При потере связи с Nav2 или ROS2 активируется узел fallback_nav_node, реализующий Bug0 (см. §3).

Условия активации:

- Отсутствие cmd_vel от Nav2 более 2 с
- Потеря MQTT-связи с лэптопом
- Явная команда от FSM

6.3. Watchdog Node (watchdog_node.py)

Сторожевой таймер мониторит:

- Heartbeat каждого узла (период 5 с, таймаут 15 с)
- Напряжение батареи (порог $V_{warn} = 6.8$ В, $V_{crit} = 6.2$ В)
- Температуру CPU (порог 80°C)
- Значение УЗ-дальномера (stuck detection)

При критическом событии: аварийная остановка + публикация log/events.

6.4. ZUPT как функция безопасности

ZUPT (Zero-velocity Update, §4.1) выполняет двойную функцию:

1. Калибровка смещений гироскопа и акселерометра в покое.
2. Предотвращение дрейфа одометрии при длительных остановках.

Без ZUPT накопленный дрейф за 60 с покоя мог бы составить > 0.5 м по положению.

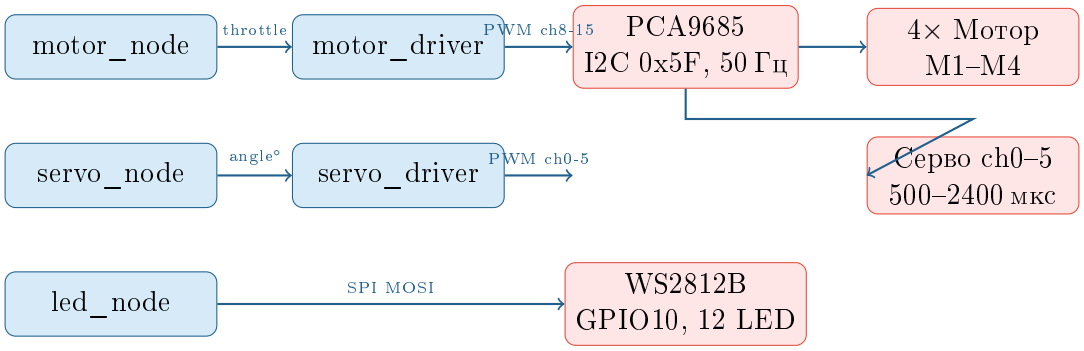
6.5. Обнаружение препятствий и обновление карты

Данные УЗ-дальномера используются одновременно для:

- Реактивного торможения (Collision Guard, §6.1)
- Обновления log-odds карты (slam_map_node, §9)

7. Блок 6: Исполнительный уровень

7.1. Архитектура исполнительного уровня



7.2. PCA9685 — ШИМ-контроллер (pca9685_driver.py)

PCA9685 — 16-канальный ШИМ-контроллер на шине I2C (адрес 0x5F).

7.2.1. Параметры ШИМ

$$f_{PWM} = 50\ \text{Гц}, \quad T_{PWM} = 20\ \text{мс} \tag{63}$$

$$\text{ON_tick} = 0, \quad \text{OFF_tick} = \text{round}\left(\frac{t_{pulse}}{T_{PWM}} \cdot 4096\right) \tag{64}$$

Диапазон счётчика: 12 бит ($0 \ldots 4095$), разрешение $\approx 4.9\ \mu$с.

7.2.2. Преобразование дроссель → ШИМ

Для мотора с управлением SLOW_DECAY:

$$\text{throttle} \in [-1.0,\ +1.0] \tag{65}$$

$$\text{duty}_{IN1} = \begin{cases} |\text{throttle}| & \text{throttle} > 0 \\ 0 & \text{иначе} \end{cases}, \quad \text{duty}_{IN2} = \begin{cases} 0 & \text{throttle} > 0 \\ |\text{throttle}| & \text{иначе} \end{cases} \tag{66}$$

7.2.3. Назначение каналов

Канал	Мотор	Позиция	IN1	IN2
14, 15	M1	Правый перед	ch15	ch14
12, 13	M2	Левый перед	ch12	ch13
10, 11	M3	Левый зад	ch11	ch10
8, 9	M4	Правый зад	ch8	ch9
		Сервоприводы		
0	—	Захват (claw)	0°–180°	500–2400 мкс
4	—	Голова (head)	90° центр	500–2400 мкс
1–3	—	Рука (arm)	суставы 1–3	500–2400 мкс

7.3. Servo Driver (servo_driver.py)

7.3.1. Преобразование угол → длительность импульса

$$t_{pulse}(\varphi) = t_{min} + \frac{\varphi}{180^\circ}(t_{max} - t_{min}) \tag{67}$$

$$t_{min} = 500\ \mu\text{c}, \quad t_{max} = 2400\ \mu\text{c} \tag{68}$$

7.3.2. Авто-отключение (Anti-buzz)

После исполнения команды через $t_{hold} = 0.5\,\text{c}$:

$$\text{duty} \leftarrow 0 \quad (\text{отключить ШИМ}) \tag{69}$$

Предотвращает постоянный гул при удержании позиции.

7.4. LED Driver (led_driver.py)

WS2812B NeoPixel через SPI MOSI (GPIO 10, Board pin 19):

- 12 адресуемых RGB-светодиодов (4 панели × 3)
- Цветовой порядок GRB

Эффекты: blink, pulse (плавное нарастание яркости), rainbow_cycle (вращение оттенка).

8. Датчики

8.1. IMU — MPU6050 (imu_node.py)

Параметр	Значение	Параметр	Значение
I2C адрес	0x68	Частота дискретизации	50 Гц
Диапазон гир.	$\pm 250°/\text{с}$	Диапазон акс.	$\pm 4\text{g}$
EMA гир.	$\alpha = 0.3$	EMA акс.	$\alpha = 0.2$
Калибровка	200 выборок	Время калибровки	$\approx 4\,\text{с}$

8.1.1. Калибровка смещений

$$\mathbf{b}_{gyro} = \frac{1}{N}\sum_{i=1}^{N} \boldsymbol{\omega}^{(i)}, \quad N = 200 \tag{70}$$

$$\mathbf{g}_{body} = \frac{1}{N}\sum_{i=1}^{N} \mathbf{a}^{(i)}, \quad \phi_0 = \arctan\left(\frac{\bar{a}_y}{\bar{a}_z}\right), \quad \vartheta_0 = \arctan\left(\frac{-\bar{a}_x}{\sqrt{\bar{a}_y^2 + \bar{a}_z^2}}\right) \tag{71}$$

8.1.2. EMA фильтрация

$$\tilde{x}[k] = (1-\alpha)\,\tilde{x}[k-1] + \alpha\, x[k] \tag{72}$$

Частота среза: $f_c = \frac{\alpha \cdot f_s}{2\pi(1-\alpha)}$. Для $\alpha = 0.2$, $f_s = 50\,\text{Гц}$: $f_c \approx 2\,\text{Гц}$.

8.2. Ультразвуковой дальномер HC-SR04 (ultrasonic_node)

8.2.1. Принцип измерения

$$d = \frac{c_{sound} \cdot t_{echo}}{2}, \quad c_{sound} \approx 343\ \text{м/с} \tag{73}$$

Диапазон: $d \in [0.02;\ 2.0]\,\text{м}$. Угол раскрытия: $\pm 7.5°$ ($\approx 15°$ полный конус).

8.3. Камера CSI (camera_node)

Параметр	Значение
Разрешение	640×480 пикс.
Частота кадров	20 Гц
Формат	JPEG (бинарный MQTT)
Фокусное расстояние	$f = 500\,\text{пикс.}$
Поле зрения	$\approx 60°$

8.3.1. Моно-оценка дальности до объекта

По угловому размеру ограничивающего прямоугольника (small-angle approx.):

$$\varphi_{obj} = 2\arctan\left(\frac{h_{px}}{2f}\right) \approx \frac{h_{px}}{f} \tag{74}$$

$$d_{est} = \frac{H_{real}}{\varphi_{obj}} = \frac{H_{real} \cdot f}{h_{px}} \tag{75}$$

Для мяча $H_{real} = 0.04$ м: $d_{est} = \frac{0.04 \cdot 500}{h_{px}}$ м.

9. SLAM и картография (slam_map_node.py)

9.1. Log-odds occupancy grid

Карта представлена сеткой 200×200 ячеек с разрешением 0.05 м (площадь 10×10 м):

$$L(m_i) = \log \frac{p(m_i = occ)}{p(m_i = free)} \tag{76}$$

9.1.1. Обновление ячеек

$$L_k(m_i) = L_{k-1}(m_i) + \Delta L(z, r, m_i) \tag{77}$$

$$\Delta L = \begin{cases} l_{occ} = +0.85 & \text{луч заканчивается в ячейке (препятствие)} \\ l_{free} = -0.40 & \text{луч проходит через ячейку (свободно)} \end{cases} \tag{78}$$

Ограничение: $L(m_i) \in [-2.0;\ +3.5]$.

9.1.2. Порог занятости

$$\hat{m}_i = \begin{cases} \text{занято} & L(m_i) > 0.6 \\ \text{свободно} & L(m_i) < -0.5 \\ \text{неизвестно} & \text{иначе} \end{cases} \tag{79}$$

9.2. Трассировка лучей (Ray casting, HC-SR04)

УЗ-конус моделируется N_{beam} лучами с полуугловым шагом:

$$\theta_j = \theta_{robot} + \frac{j \cdot \mathrm{FOV}_{half}}{N_{beam}/2}, \quad j \in \{-N/2, \ldots, N/2\} \tag{80}$$

Для каждого луча: Bresenham line rasterization от позиции робота до точки $r \cdot (\cos\theta_j, \sin\theta_j)$.

Координаты ячейки:

$$c_i = \left\lfloor \frac{p_i - p_{origin}}{d_{cell}} \right\rfloor, \quad d_{cell} = 0.05 \text{ м} \tag{81}$$

9.3. Реестр объектов

Обнаруженные YOLO-объекты хранятся с мировыми координатами:

- Кластеризация: слияние наблюдений ближе 0.30 м
- Устаревание: удаление не виденных 120 с
- Минимальная уверенность: conf ≥ 0.35

10. Система восприятия (yolo_detector_node.py)

10.1. YOLOv11n — архитектура и параметры

Параметр	Значение
Модель	YOLOv11n (nano) / YOLOv8n
Экспорт	ONNX Runtime
Входной размер	416×416 пикс.
Порог уверенности	$conf_{thresh} = 0.40$
NMS IoU	$iou_{thresh} = 0.45$
Ускорение	CUDA (если доступно) / CPU
Потоки CPU	$\lfloor N_{cpu}/2 \rfloor$ (intra-op)

10.2. Конвейер инференса

$$I_{JPEG} \xrightarrow{\text{cv2.imdecode}} I_{BGR} \xrightarrow{\text{resize}} I_{416} \xrightarrow{/255} I_{norm} \xrightarrow{\text{CHW}} \mathbf{x}_{onnx} \xrightarrow{\text{ONNX}} \mathbf{y} \xrightarrow{\text{NMS}} \{bbox_i\} \tag{82}$$

10.3. HSV-классификация цвета мяча

Для каждого ограничивающего прямоугольника:

$$I_{HSV} = \text{cvtColor}(I_{ROI},\ \text{BGR2HSV}) \tag{83}$$

$$M_{colour} = \sum_i \text{inRange}(I_{HSV},\ lo_i,\ hi_i) \tag{84}$$

Цвет	H	S	V
Красный	[0–10] [160–180]	70–255	70–255
Синий	[85–135]	60–255	60–255
Зелёный	[35–85]	60–255	60–255
Жёлтый	[22–38]	80–255	80–255
Оранжевый	[10–25]	80–255	80–255
Белый	[0–180]	0–40	180–255
Чёрный	[0–180]	0–255	0–60

Победитель — цвет с максимальной долей маски M_{colour} (минимум 15% пикселей ROI).

11. Сводная таблица параметров системы

Параметр	Символ	Значение	Источник
Геометрия робота			
База колёс	L	0.17 м	config.yaml
Радиус робота	r_{robot}	0.12 м	config.yaml
Ширина арены	W_a	3.0 м	config.yaml
Кинематика			
Макс. линейная скорость	v_{max}	0.30 м/с	config.yaml
Макс. угловая скорость	ω_{max}	2.0 рад/с	config.yaml
Скорость патрулирования	v_{patrol}	0.20 м/с	config.yaml
IMU EKF			
Шум процесса (угол)	q_ψ	0.001	ekf_imu.py
Шум процесса (смещение)	q_b	0.0001	ekf_imu.py
Шум измерения (акс.)	r_{acc}	0.5	ekf_imu.py
Порог ZUPT (вход)	ε_{enter}	0.015 рад/с	ekf_imu.py
Порог ZUPT (выход)	ε_{exit}	0.040 рад/с	ekf_imu.py
Шум ZUPT	r_{zupt}	10^{-4}	ekf_imu.py
Velocity EKF			
Постоянная времени мотора	τ_{motor}	0.25 с	config.yaml
Постоянная времени торм.	τ_{brake}	0.08 с	config.yaml
Шум скорости	q_v	0.20	velocity_ekf.py
Шум смещения акс.	q_{ba}	0.003	velocity_ekf.py
Шум акселерометра	r_{acc}	0.06	velocity_ekf.py
Precision Drive			
Скорость движения	v_{drive}	0.12 м/с	config.yaml
Медленная зона	v_{slow}	0.06 м/с	config.yaml
Ползучая зона	v_{crawl}	0.04 м/с	config.yaml
Точность прибытия	ε_{pos}	5 мм	config.yaml
Точность курса	ε_θ	0.02 рад	config.yaml
K_P (курс)	K_P	2.5	config.yaml
K_I (курс)	K_I	0.3	config.yaml
Collision Guard			
Дистанция полной остановки	r_{stop}	0.20 м	motor_node.py
Дистанция начала торм.	r_{warn}	0.40 м	motor_node.py
SLAM			

Продолжение таблицы параметров

Параметр	Символ	Значение	Источник
Разрешение карты	d_{cell}	0.05 м	config.yaml
Размер карты	—	200×200	config.yaml
Log-odds занято	l_{occ}	$+0.85$	config.yaml
Log-odds свободно	l_{free}	-0.40	config.yaml
Клиппинг	L_{clip}	$[-2.0;\ +3.5]$	slam_map_node.py
YOLO			
Входной размер	—	416×416	yolo_detector.py
Порог уверенности	$conf_{th}$	0.40	config.yaml
NMS IoU	iou_{th}	0.45	yolo_detector.py

12. Топология MQTT-топиков

Топик	Откуда	Куда / Назначение
.../odom	motor_node	EKF, SLAM, dashboard
.../imu	imu_node	motor_node, EKF
.../camera	camera_node	YOLO, dashboard
.../range	ultrasonic	Collision Guard, SLAM
.../battery	battery_node	watchdog, dashboard
.../temperature	temp_node	watchdog, dashboard
.../status	fsm_node	dashboard (retained)
.../cmd_vel	fsm/Nav2	motor_node (auton.)
.../cmd_vel/manual	dashboard/app	motor_node (ручной)
.../voice_command	Android/Vosk	fsm_node
.../ball_detection	YOLO/laptop	fsm_node
.../slam_map	slam_map_node	dashboard, Nav2-bridge
.../goal_pose	fsm_node	Nav2 (через мост)
.../claw/command	fsm_node	servo_node
.../head/command	dashboard/fsm	head_node
.../arm/command	dashboard/fsm	arm_node
.../led/command	fsm_node	led_node
.../precision_drive/command	dashboard	precision_drive_node
.../path_recorder/command	fsm/dashboard	path_recorder_node
.../patrol/command	fsm/dashboard	patrol_node
.../follow_me/command	dashboard/fsm	follow_me_node
.../log/events	все узлы	dashboard (лог)
.../perf/{node}	все узлы	мониторинг задержек

13. Взаимодействие уровней: полный цикл управления

Рассмотрим полный цикл от команды «найди мяч» до физического движения:

1. Android/Dashboard: Пользователь произносит «найди мяч». Vosk offline-распознаёт текст и публикует в samurai/robot1/voice_command: "найди мяч".
2. FSM (Блок 1): Regex-паттерн получинайди| срабатывает. FSM переходит в состояние SEARCHING. Публикуется cmd_vel: {linear: 0, angular: 0.5} (поиск вращением).
3. YOLO (Блок 2/лэптоп): Кадры камеры анализируются YOLOv11n. При обнаружении мяча публикуется ball_detection: {x:180, y:240, w:40, h:42, colour:"red"}.
4. FSM: Переход SEARCHING → TARGETING. Вычисляется $e_x = -0.44$. Команда $\omega = +0.35$ рад/с.
5. Precision Drive / Motor Node (Блок 4): PI-регулятор курса поддерживает $e_\theta \to 0$. cmd_vel: {linear: 0.12, angular: 0.35} → motor_driver.
6. Collision Guard (Блок 5): УЗ-дальномер = 0.85 м. $\lambda(0.85) = 1.0$ — ограничений нет.
7. PCA9685 (Блок 6): Дроссель L=0.175, R=0.065 → ШИМ ch8–15.
8. Датчики (обратная связь): IMU @ 50 Гц возвращает новый ψ. Odometry @ 20 Гц обновляет (x, y, θ). EKF корректирует оценку состояния.
9. Цикл повторяется: до выполнения захвата и возврата.

14. Заключение

В настоящем отчёте описана полная архитектура системы управления автономного гусеничного робота Samurai, реализующая шестиуровневую иерархию от постановки задачи до физического движения двигателей.

Ключевые архитектурные решения:

1. MQTT без ROS2 на Pi — устраняет overhead DDS, уменьшает RAM с ≈ 00 ÌÁäî≈ 0 ÌÁ.

1. Гибридный EKF — раздельные фильтры для угла (2 состояния) и скорости (2×ñîñòî„íè„)ïðîùåäèíãî6 – DOFïðèñîîîñòàâèìéòî⊲íîñòèäë„ïëîñêîãîäâè_åíè„.

1. Log-odds SLAM на борту Pi — лёгкая реализация без зависимостей от ROS2 SLAM Toolbox.
2. PI + боковой корректор вместо полного MPC — достаточно для достижения точности 5 мм в сценариях precision_drive.
3. ZUPT в обоих EKF — критично для предотвращения дрейфа при паузах FSM и длительных остановках.
4. Fallback Bug0 — гарантирует базовую навигацию без лэптопа.
5. Трёхуровневый Collision Guard — мгновенная реакция на препятствия без участия FSM или Nav2.

0.3cm]$SamuraiRobotProject$ | Äîêóìåíòñîçäàí : àïðåëü2026 0.1cm]Âñåèõîäíûåêîäû : pi_nodes/, compu